

Task 63: Shortest Path with Alternating Colors

Problem Description

You are given an integer n , the number of nodes in a directed graph where the nodes are labeled from 0 to $n - 1$. Each edge is red or blue in this graph, and there could be self-edges and parallel edges. You are given two arrays `redEdges` and `blueEdges` where: `redEdges[i] = [ai, bi]` indicates that there is a directed red edge from node a_i to node b_i in the graph, and `blueEdges[j] = [uj, vj]` indicates that there is a directed blue edge from node u_j to node v_j in the graph. Return an array `answer` of length n , where each `answer[x]` is the length of the shortest path from node 0 to node x such that the edge colors alternate along the path, or -1 if such a path does not exist.

Java Solution

```

class Solution {
    public int[] shortestAlternatingPaths(int n, int[][] redEdges, int[][] blueEdges) {
        List<Integer>[][] graph = new ArrayList[2][n];
        for (int i = 0; i < n; i++) {
            graph[0][i] = new ArrayList<>();
            graph[1][i] = new ArrayList<>();
        }
        for (int[] e : redEdges) graph[0][e[0]].add(e[1]);
        for (int[] e : blueEdges) graph[1][e[0]].add(e[1]);

        int[][] dist = new int[2][n];
        for (int i = 0; i < 2; i++) Arrays.fill(dist[i], -1);

        Queue<int[]> q = new LinkedList<>();
        q.offer(new int[]{0, 0});
        q.offer(new int[]{0, 1});
        dist[0][0] = 0;
        dist[1][0] = 0;

        while (!q.isEmpty()) {
            int[] curr = q.poll();
            int node = curr[0], color = curr[1];
            int nextColor = 1 - color;

            for (int neighbor : graph[nextColor][node]) {
                if (dist[nextColor][neighbor] == -1) {
                    dist[nextColor][neighbor] = dist[color][node] + 1;
                    q.offer(new int[]{neighbor, nextColor});
                }
            }
        }

        int[] ans = new int[n];
        for (int i = 0; i < n; i++) {
            int d1 = dist[0][i], d2 = dist[1][i];
            if (d1 == -1 && d2 == -1) ans[i] = -1;
            else if (d1 == -1) ans[i] = d2;
            else if (d2 == -1) ans[i] = d1;
            else ans[i] = Math.min(d1, d2);
        }
        return ans;
    }
}

```